

课程笔记

CS146S Week 4: Coding Agent Patterns

基于 Mihail Eric + Boris Cherny (Claude Code) 授课内容整理

2025 年 10 月

课程: Stanford CS146S

学期: Fall 2025

讲次: 2 lectures (Mon + Fri)

网站: <https://themodernsoftware.dev>

项目主页: <https://github.com/hqhq1025/ai-course-notes>

目录

1 如何成为 Agent Manager	2
1.1 软件开发团队的演进	2
1.2 软件任务的分解	2
1.3 指导 Agent 的四种技术	2
1.3.1 Agent 行为文件	2
1.3.2 Hooks	3
1.3.3 Commands	3
1.3.4 Subagents	3
1.4 最佳实践	3
1.5 本章小结	4
2 嘉宾讲座：Claude Code 的设计哲学 (Boris Cherny)	4
2.1 编程正处于拐点	4
2.2 Claude Code 的设计原则	4
2.3 一个 Claude Code，多种形态	4
2.4 Claude Code 的典型使用模式	4
2.4.1 代码库问答与调研	4
2.4.2 工作流适配任务	5
2.4.3 快速原型迭代	5
2.5 管理多个 Agent 时的组织约束	5
2.6 本章小结	5
3 总结与延伸	5
3.1 关键点	6
3.2 落地清单	6
3.3 角色分工对照表	6
3.4 团队采用成熟度模型	6
3.5 拓展阅读	7

1 如何成为 Agent Manager

1.1 软件开发团队的演进

软件开发团队的组织形态正在经历第四次重大转型：

1. **1940–1960s**：单个开发者管理整个项目
2. **1970–1990s**：软件团队主流化，专业分工出现
3. **2023–2025**：开发者借助 AI 编程系统辅助工作
4. **2025+**：单个开发者管理多个 AI Agent 的工作输出

从管理人到管理 Agent

开发的演化路径是：管理自己的输出 → 管理团队的输出 → 管理 AI 辅助团队的输出 → **单人管理多个 AI Agent 的输出**。最后一步是当前正在发生的变革。

1.2 软件任务的分解

一个典型的软件开发任务可以分解为以下步骤，其中绿色表示人类主导，蓝色表示 Agent 可以承担：

1. 提供高层需求 [人类]
2. 将需求转化为设计文档 [人类/Agent]
3. 根据设计文档实现方案 [Agent]
4. 添加测试 [Agent]
5. 确保 CI 通过 [Agent]
6. 代码审查 [Agent]
7. 更新文档 [Agent]

人类的核心价值在第 1–2 步

在 Agent 管理模式下，人类工程师的最高价值体现在**需求理解**和**架构设计**上。这两步需要业务上下文、技术判断力和“品味”——恰好是当前 AI 最薄弱的环节。

1.3 指导 Agent 的四种技术

1.3.1 Agent 行为文件

如 CLAUDE.md、.cursorrules、AGENTS.md 等，提供项目级别的行为指导。

1.3.2 Hooks

确定性脚本，在预定义的事件触发时运行：

- PreToolUse: 工具调用前
- PostToolUse: 工具调用后
- UserPromptSubmit: 用户提交 prompt 时
- PreCompact: context 压缩前

1.3.3 Commands

将常用 prompt 保存为文件，Agent 可以按需执行。典型用例：运行测试、审查代码、提交 git commit 并推送。

1.3.4 Subagents

运行时委派机制，核心目的：

- 为不同类型的工作创建独立的开发者“人格”（frontend、backend 等）
- 清晰分离不同工作流的 context
- 提供定制化的 system prompt、工具集和独立的 context window

Subagent 是 Agent 管理 Agent 的雏形

Subagent 模式代表了一个重要趋势：Agent 管理其他 Agent。主 Agent 负责任务分解和委派，subagent 专注于特定领域的执行，每个 subagent 拥有独立的 context 避免信息过载。

1.4 最佳实践

1. **设置严格的防护栏**：代码库中的测试 + CI/CD 最佳实践是不可或缺的 backstop
2. **审计每个 Agent 的修改**：标记 (label) 每个由 Agent 产生的 diff
3. **不同任务使用不同模型**：简单任务用快速模型，复杂任务用强模型
4. **复杂任务需要更多前期引导**：不是所有任务都适合完全异步委派
5. **频繁 checkpoint (commit)**：定期提交代码，防止 Agent 偏离方向时丢失进展

开放问题

- 如何自动化每个任务最初 10-20% 的调研阶段？
- 如何维护一个待执行任务的队列？（对于一次性修改较容易，但对持续性工作流仍是挑战）

1.5 本章小结

Agent Manager 模式要求开发者从“写代码的人”转型为“管理写代码的 Agent 的人”。核心技术包括行为文件、hooks、commands 和 subagents。关键原则是：设置防护栏、审计修改、匹配任务与模型、频繁 checkpoint。

2 嘉宾讲座：Claude Code 的设计哲学 (Boris Cherny)

2.1 编程正处于拐点

Claude Code 的创造者 Boris Cherny 分享了一个关键洞察：编程语言和 IDE 的生产力都在沿指数曲线增长，而 AI 正是驱动最新一波增长的核心力量。

编程生产力的指数增长

从 Assembly 到 FORTRAN，从 C 到 Python，从 VS Code 到 Cursor/Claude Code——每一代编程语言和 IDE 都在对数尺度上实现了生产力的线性提升。验证（verification）技术也在同步演进：从手动调试到静态类型，从自动化测试到 AI 驱动的 fuzz testing 和 self-play。

2.2 Claude Code 的设计原则

Claude Code 的核心设计哲学是三个“无处不在”：

1. **Terminal-native**：在终端中运行，而非 IDE 插件
2. **Low-level model access**：提供对底层模型的直接访问
3. **Infinitely hackable**：无限可定制

2.3 一个 Claude Code，多种形态

Claude Code 以多种形态服务不同场景：

- **Terminal**：命令行直接使用
- **IDE**：集成到 VS Code 等编辑器
- **Web & iOS**：通过 `claude.ai/code` 访问
- **GitHub App**：通过 `/install-github-app` 安装
- **SDK**：程序化调用，支持 JSON 输出、管道组合

2.4 Claude Code 的典型使用模式

2.4.1 代码库问答与调研

- “How do I make a new ValidationTemplateFactory?”
- “Why did we fix issue #18363 by adding the if/else in login.ts?”
- “Look at PR #9383, then verify which app versions were impacted”
- “What did I ship last week?”

2.4.2 workflow 适配任务

不同任务适合不同的 workflow 模式：

- **explore** → **plan** → **confirm** → **code** → **commit**：适合需要调研的 bug fix
- **tests** → **commit** → **code** → **iterate** → **commit**：TDD（测试驱动开发）模式
- **code** → **screenshot** → **iterate**：UI 开发的视觉迭代模式

2.4.3 快速原型迭代

Boris 展示了一个生动的原型迭代案例：通过连续的自然语言指令，快速迭代 TODO 列表的 UI 设计——从最初的工具使用显示，到内联标题，到独立面板，再到与 spinner 合并显示，每次迭代只需一条 prompt。

为 6 个月后的模型而构建

Boris 强调的核心理念：不要为今天的模型能力设计你的 workflow，而要为 **6 个月后的模型能力**做准备。模型在快速进化，你的工具和工作流也应该具备演进能力。

2.5 管理多个 Agent 时的组织约束

Week 4 的另一个隐含结论是：当开发者开始同时管理多个 Agent，项目管理方式本身也要变化。

- **任务必须可切分**：如果需求本身模糊不清，就很难安全委派给多个 Agent 并行执行
- **边界必须清晰**：每个 Agent 负责哪些文件、哪些验证步骤、哪些提交粒度，需要事先说清
- **审查必须标准化**：越依赖 Agent 并行，越需要一致的 diff 审查和回滚机制

Agent Manager 的主要失败模式不是模型不聪明，而是任务切分不合理

很多并行失败并非因为 Agent 做不到，而是因为人类在开始前没有给出清晰的责任边界和验收标准。换言之，Agent Manager 首先是项目管理能力，然后才是 prompt 技巧。

2.6 本章小结

Claude Code 的设计哲学是 terminal-native、低层模型访问和无限可定制。它覆盖了从代码库调研到 CI/CD 管理的整个 SDLC。核心使用策略是根据任务类型选择合适的工作流模式，并为未来的模型能力提前做好准备。

3 总结与延伸

Week 4 从两个互补视角探讨了 Coding Agent 的使用模式：Mihail Eric 讲解了 Agent Manager 的方法论和管理技术，Boris Cherny 则从 Claude Code 创造者的角度分享了工具设计哲学和实践经验。

3.1 关键点

- 软件开发正从“人管人”转向“人管 Agent”——这是第四次组织形态转型
- 四种 Agent 指导技术：行为文件、hooks、commands、subagents
- 关键实践：防护栏 (tests + CI/CD)、审计标记、模型匹配、频繁 checkpoint
- Claude Code 设计哲学：terminal-native、low-level access、infinitely hackable
- 根据任务类型选择工作流：explore-plan-code、TDD、视觉迭代
- 为 6 个月后的模型能力做准备

3.2 落地清单

1. 为常见任务建立可复用的 commands、hooks 和行为文件
2. 用 subagents 处理明确分工的并行任务，而不是让所有工作挤进一个上下文
3. 把测试、CI 和人工审查当作 Agent 工作流的硬边界，而非事后补救

3.3 角色分工对照表

角色	主要职责	失败时的补救方式
人类 Agent Manager	定义目标、拆分任务、设置边界、做最终取舍	重新划分任务、补充约束、决定回滚或继续
执行型 Agent	阅读上下文、修改代码、运行测试、生成提交	通过 hooks / tests / review 暴露问题并重试
审查与 CI 系统	提供客观验证、阻止明显错误进入主干	用标准化检查和可回放日志减少隐性风险

表 1: Agent Manager 模式下的人机职责边界

3.4 团队采用成熟度模型

从课程内容可以提炼出一个很实用的 adoption 路径：

1. **个人试验期**：个别工程师用 Agent 处理问答、脚手架代码和局部重构
2. **团队规范期**：开始沉淀 commands、hooks、review 规则和统一的行为文件
3. **并行协作期**：Agent 被用于明确分工的多线程任务，开发者开始承担 Agent Manager 角色
4. **系统整合期**：Agent 工作流与 CI、代码审查、知识库和项目管理工具深度打通

成熟度提升的标志不是“用了更多 Agent”，而是“失败更可控”

真正进入成熟阶段的团队，不是看上去更炫，而是当 Agent 出错时更容易被测试、hooks、代码审查和责任边界及时捕获。这也是为什么 Boris 和 Mihail 都反复强调 guardrails，而不是只强调速度。

3.5 拓展阅读

- Claude Code 官网: <https://claude.ai/code>
- Claude Code SDK 文档: <https://docs.anthropic.com/en/docs/claude-code>
- Awesome Claude Agents: <https://github.com/vijaythecoder/awesome-claude-agents>
- SuperClaude Framework: https://github.com/SuperClaude-Org/SuperClaude_Framework